

# Accommodation Rental System

## About This Task

Before the technical interview you build a small app for renting out places to stay. At the interview we open your code together and talk through the decisions you made.

The task is also the way into the technical interview. The scope is deliberately limited, but we expect what you do build to be built well — with tests, a clear structure and a usable UI.

Feel free to take UI inspiration from Airbnb or Booking. We are not looking for visual design; we are looking for something that can actually be used.

Anything not specified here is your choice.

## Stack

NestJS, React, PostgreSQL, Docker.

## What to Build

### Portal

The public side, one per tenant.

1. A list of all listings.
2. Opening a single listing.
3. Filtering the list by city, number of guests, price range and date range. The date filter returns only listings that are free across the *whole* range.
4. On a single listing — when that listing is available to book.

### Accounts and Access

5. Registration and sign-in. Registration is for clients — host accounts are created by the superadmin.
6. Three kinds of user — client, host and superadmin:
  - a **client** searches and views listings;
  - a **host** manages the listings and calendar of their tenant;
  - a **superadmin** creates tenants, configures them and adds host accounts to them.

Keep auth simple — we are not asking for SSO, 2FA or refresh-token rotation. What interests us is how you separate **who you are** from **what you may do**.

## Host Panel

7. Editing listings.
8. Blocking individual days in the calendar for a listing.
9. Viewing bookings.

## Admin Panel

10. Creating, editing and deleting tenants.
11. Tenant configuration. Name and slug are required; anything beyond that is up to you — logo, currency, primary colour, contact email.
12. Adding host accounts to a tenant. A tenant may have several hosts.

## Multi-Tenancy

A tenant is one portal at `/ {tenant-slug}`, with its own configuration and one or more host accounts.

The superadmin creates tenants through the admin panel. One tenant in the data is enough, but adding a new one has to work.

One tenant's data must never be visible on another tenant's portal. How you enforce that — in the database, in the queries, or somewhere else entirely — is your decision.

What happens to a client's account across portals is also yours to decide. Remember what you chose; we will ask.

## Tests

Tests are required, the choice is yours.

## What You Get

- **contracts.ts** — the shape of the data. The comments explain how dates and booking statuses are read.
- **listings.csv** — 1,000 listings.
- **bookings.csv** — 12,757 bookings.

How you split the 1,000 listings across tenants and hosts is up to you.

For users, tenants and blocked days you get nothing — no data and no types. Modelling those is part of the task.

How you load the data into the database is your choice.

## Not Needed

- Booking and payment. Clients do not book — the bookings already exist in `bookings.csv` and the host only views them.
- Maps.
- Deployment.

## A Few Notes

- If you did not get to something, or left it out on purpose, tell us at the interview. That is as good a thing to talk about as a finished screen.
- If something is unclear, either ask us or make an assumption and remember what you assumed.
- Use the tools you normally use, AI included.
- We do not expect everything to be finished. If you have to choose, tell us what you chose and why.

## Submitting and Deadline

- The deadline is **three to five days**. We expect you to use the tools you normally work with, AI included. If you need more time, get in touch with HR and we will sort it out.
- Send **the whole repository** to HR — a GitHub or GitLab or any other git repository host is fine.
- We schedule the technical interview once we have it.